

PROJECT REPORT

Personal Finance Manager

*Design, implementation and operation of a
self-hosted finance application*



A guide to the whole process: from a set of desktop scripts
to a deployed web application with a Telegram bot,
and from a private repository to a public one.

Ismael Sallami Moreno

July 25, 2026

Source code: <https://github.com/Ismael-Sallami/personal-finance-manager>

Built with FastAPI, SQLAlchemy, Alembic, Postgres and the Telegram Bot API.

Contents

1	Introduction	4
1.1	What this report is	4
1.2	Who it is for	4
1.3	Where the project came from	4
1.4	What the application does today	4
1.5	Design principles	5
1.6	How to read this report	5
2	Architecture	6
2.1	The shape of the application	6
2.2	Directory layout	6
2.3	The rule that keeps it tidy	6
2.4	The service modules	7
2.5	Configuration	7
2.6	The application lifespan	8
2.7	Two health endpoints, on purpose	8
3	The data model	9
3.1	Seven tables	9
3.2	Money is never a float	9
3.3	Transactions	9
3.3.1	Why year and month are stored separately	10
3.3.2	Why the category is a string	10
3.4	Investments	10
3.5	Bank accounts and net worth snapshots	11
3.6	Migrations	12
4	Expenses and income	13
4.1	The problem with data entry	13
4.2	Parsing a pasted block	13
4.3	Evaluating arithmetic without eval	14
4.4	The preview step	14
4.5	Reading amounts from a form	15
4.6	The two views	15
4.7	Investing is spending, and also not	15
5	Investments	17
5.1	Three brokers, three formats	17
5.2	One shape for all of them	17
5.3	Reading each format	18
5.3.1	MyInvestor	18
5.3.2	Kraken	18
5.3.3	Trade Republic	18
5.4	Profit depends on the broker	19
5.5	Valuation: manual first	20

5.6	From an ISIN to a ticker	20
5.7	Contributions	21
6	Net worth and aggregation	22
6.1	What was missing	22
6.2	Accounts	22
6.3	The aggregation module	22
6.4	Month ranges without special cases	23
6.5	The dashboard	23
6.6	An empty database is a valid state	24
7	Reports	25
7.1	Why reports at all	25
7.2	Excel	25
7.3	PDF	25
7.4	A bug worth keeping	26
7.5	Percentages that do not lie	26
7.6	Lazy imports, again	26
8	The Telegram bot	28
8.1	Why a bot	28
8.2	No extra dependency	28
8.3	Webhook instead of polling	28
8.4	Two locks on the door	29
8.5	Confirm before writing	29
8.6	Periods and ranges	29
8.7	Commands	30
8.8	Synchronous and asynchronous sending	30
9	Keeping it running	32
9.1	The failure that started this chapter	32
9.2	Three pieces	32
9.3	Why the health endpoints differ	32
9.4	Scheduled jobs, twice	33
9.5	Two ways to send the token	33
9.6	Backups	34
10	Security	36
10.1	The threat model	36
10.2	Passwords	36
10.3	Sessions	36
10.4	CSRF tokens	37
10.5	Slowing down brute force	37
10.6	Response headers	38
10.7	Uploads and pasted text	38
10.8	Secrets and the repository	38
11	Testing	39
11.1	What is worth testing	39
11.2	The database fixture	39
11.3	Overriding dependencies	40
11.4	No network in the test suite	40

11.5	Parser tests without private data	40
11.6	The startup test	40
11.7	Regression tests carry their reason	41
11.8	Continuous integration	41
12	Deployment	42
12.1	Two paths, both supported	42
12.2	The image	42
12.3	Compose	42
12.4	Managed hosting	43
12.5	Connecting to Postgres	43
12.6	Migrations in production	44
12.7	The first-boot checklist	44
13	Publishing a private project	45
13.1	The exercise	45
13.2	Step one: decide what is personal	45
13.3	Step two: a fresh history	45
13.4	Step three: the translation	46
13.5	Step four: collapse the migrations	47
13.6	Step five: rewrite the documentation	47
13.7	Step six: verify, do not assume	47
13.8	A checklist	48
14	Lessons	49
14.1	What the project taught	49
14.1.1	Correctness in money is a type decision	49
14.1.2	The interface is not the application	49
14.1.3	Boot time is a feature	49
14.1.4	Choose where to fail loudly	49
14.1.5	Free tiers are a design constraint	49
14.1.6	Error messages should say what to do	49
14.1.7	Small representation changes remove whole bug families	50
14.2	What would be done differently	50
14.3	Closing	50

Introduction

1.1 What this report is

This report explains a complete software project from end to end: what problem it solves, how it was designed, how each part works, how it is deployed, and how it is kept running once nobody is watching it. It is written as a guide, so a reader who has never seen the code can follow the reasoning and learn from it.

The project is a **personal finance manager**: a small web application that records expenses and income, tracks investments across several brokers, calculates net worth, produces Excel and PDF reports, and answers from a Telegram chat.

Every chapter follows the same shape. It states the problem, shows the decision that was taken, and explains why the alternatives were left aside. Code appears only when it makes the explanation shorter than prose would.

1.2 Who it is for

The report assumes the reader knows Python and has seen a web framework before. It does not assume knowledge of FastAPI, SQLAlchemy, Alembic, the Telegram Bot API or financial concepts. Those are introduced where they first appear.

A reader who only wants to run the application does not need this document; the repository `README.md` covers that in a page. This report is for the reader who wants to understand *why* the code looks the way it does.

1.3 Where the project came from

The application replaced five desktop scripts written in Python with a tkinter interface. Each script covered one area: expenses, income, investments, a shared property, and account balances. They worked, but they carried the problems that small personal tools usually carry:

- They only ran with the computer switched on, so nothing could be recorded from a phone.
- Data lived in loose files, mostly spreadsheets, text files and PDFs. There was no database, no history, and no safe way to query anything.
- Amounts were handled as `float`. Cents drifted.
- Personal data and absolute paths were written directly in the source.
- There were no tests, so a change in one script could break another without anybody noticing.

The goal of the rewrite was a single application, reachable from a phone, with a real database, sound arithmetic, and no personal data in the code.

1.4 What the application does today

Area	What it covers
Expenses and income	Manual entry plus a paste importer that reads the notes the user already keeps on a phone, evaluates the arithmetic in each line, and shows a preview before saving.
Investments	Statement import for three brokers, manual positions, automatic revaluation from market prices, and recurring contributions.
Net worth	Bank accounts and cash added to the investment value, with a daily snapshot that feeds an evolution chart.
Reports	Excel exports and PDF reports with charts, for a month, a year or any range of months.
Telegram bot	Logging entries from a chat with a confirmation step, plus summaries, balances and reports on demand.
Operations	Health endpoints, scheduled jobs, and a weekly database backup delivered outside the hosting provider.

Table 1.1: Scope of the application.

1.5 Design principles

Five rules shaped almost every decision in the project. They are stated here once and referenced throughout the report.

The five rules

1. **Money is never a float.** Amounts are `Decimal` in Python and `NUMERIC` in the database.
2. **Nothing personal in the code.** Every value that changes between users comes from the environment.
3. **One place per calculation.** The web pages, the reports and the bot call the same functions, so their totals cannot disagree.
4. **Fail loudly where it matters, quietly where it does not.** A failed backup returns an error; a failed price lookup leaves the old value in place.
5. **Boot fast.** Slow startup is not a comfort issue; on a host that sleeps, it is the difference between a bot that answers and one that does not.

1.6 How to read this report

Chapters 2 to 7 describe the application from the inside: structure, data, and each functional area. Chapter 8 covers the Telegram bot. Chapters 9 to 12 cover everything that happens after the code is written: running it, securing it, testing it and deploying it. Chapter 13 describes a separate exercise, turning a private repository into a public one without leaking anything. The last chapter collects the lessons that were only obvious afterwards.

Architecture

2.1 The shape of the application

The application is a server-rendered web application. There is no separate front-end project and no JSON API between the pages and the database. A browser asks for a page, FastAPI runs a handler, the handler calls a service, and the service returns plain data that a Jinja template renders.

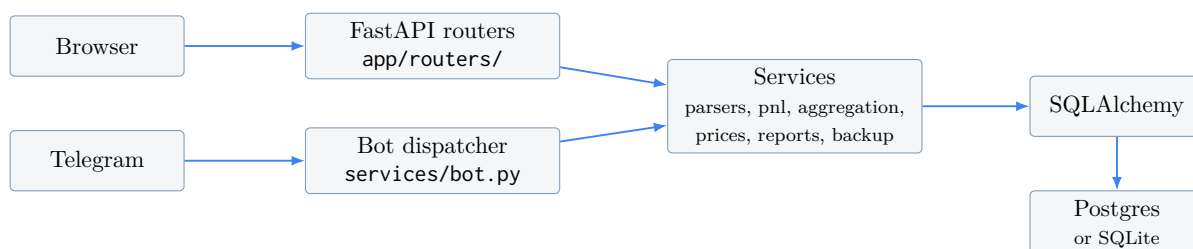


Figure 2.1: Both entry points, the web pages and the bot, share the same services.

The important line in that diagram is the one from the bot into the services. The bot is not a second implementation of the application; it is a second interface to the same functions. When a total changes, it changes in one place and every interface follows.

2.2 Directory layout

app/	
main.py	application object, middleware, lifespan, health
config.py	settings read from the environment
db.py	engine and session factory
models.py	ORM models
auth.py	login , logout , route guard
security.py	password hashing, CSRF, login rate limit
templating.py	shared Jinja instance and filters
format.py	money and percentage formatting
scheduler.py	in-process scheduled jobs
routers/	one module per section of the site
services/	every rule that decides a number
templates/	Jinja pages
static/	compiled CSS
migrations/	Alembic
scripts/	seed.py, backup.py
tests/	

2.3 The rule that keeps it tidy

Route handlers do four things and nothing else: read the form, check the CSRF token, call a service, and redirect or render. They never contain arithmetic.

The reason is not style. The Telegram bot and the PDF generator do not go through routers at all, so any calculation written inside a handler would be invisible to them and would eventually be duplicated. Duplicated arithmetic on money drifts, and drifting numbers are hard to notice and harder to trust.

Where logic lives

If a piece of code decides an amount, a total, a percentage or a category, it belongs in `app/services/`. If it decides a redirect or an HTTP status, it belongs in `app/routers/`.

2.4 The service modules

Module	Responsibility
<code>parsers/</code>	Turn a broker file into a list of positions. One module per broker, plus a dispatcher.
<code>pnl.py</code>	Profit of a position. The formula depends on the broker.
<code>aggregation.py</code>	Every total the application shows: KPIs, chart series, net worth snapshots.
<code>prices.py</code>	Market prices, and the mapping from a crypto pair to a ticker.
<code>symbol_lookup.py</code>	From an ISIN to a market symbol, with a cache.
<code>contributions.py</code>	One-off and monthly contributions.
<code>expenses_parse.py</code>	The paste parser and its safe arithmetic.
<code>reports/</code>	Excel, PDF and the charts embedded in them.
<code>bot.py</code>	Telegram: dispatch, replies, keyboards, confirmation.
<code>backup.py</code>	Database dump and delivery.

Table 2.1: The service layer.

2.5 Configuration

All configuration is read once, at import time, into a single settings object built with `pydantic-settings`:

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=APP_ROOT / ".env", extra="ignore")

    database_url: str = "" # empty -> local SQLite
    secret_key: str = "dev-insecure-change-me"
    cookie_secure: bool = False
    is_prod: bool = False
    report_owner: str = "Account owner"
    currency_symbol: str = "€"
    ...

    @property
    def sqlalchemy_url(self) -> str:
        if self.database_url.strip():
            return self.database_url.strip()
        return f"sqlite:///{'APP_ROOT' / 'dev.db'}"
```


Two properties matter more than the rest. `sqlalchemy_url` makes SQLite the default, so a fresh clone runs with no database to install. And `telegram_enabled` is simply whether a token was provided, which is what decides at boot whether the bot router is mounted at all.

Optional features are optional

The bot, the scheduler and the task endpoints are all disabled by an empty value: no token, no bot; no `TASKS_TOKEN`, no task endpoints. A user who only wants the web pages never has to switch anything off.

2.6 The application lifespan

FastAPI runs a lifespan function on startup and shutdown. Three things happen there:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    if settings.is_prod and settings.secret_key == "dev-insecure-change-me":
        raise RuntimeError("SECRET_KEY is not configured in production.")
    scheduler = None
    if settings.use_scheduler:
        from app.scheduler import start_scheduler
        scheduler = start_scheduler()
    if settings.telegram_enabled:
        from app.services.bot import setup_webhook
        await setup_webhook()
    yield
    if scheduler:
        scheduler.shutdown(wait=False)
```

The first check refuses to boot a production instance with the default signing key. A known key means any visitor can forge a session cookie, so failing to start is the correct response, not a warning in a log nobody reads.

The webhook registration is repeated on every boot on purpose. It is idempotent, and it means a redeploy is enough to recover a bot that lost its webhook.

2.7 Two health endpoints, on purpose

The application exposes `/health` and `/health/db`, and they are not interchangeable.

- `/health` answers always and never touches the database. It is the liveness check for the hosting platform.
- `/health/db` runs `SELECT 1` and answers 503 when the database does not reply. It is the readiness check for an external monitor.

Why not one endpoint

If the platform healthcheck queried the database, a brief database outage would make the platform consider the application unhealthy and tear the deployment down. The application would then be gone even after the database came back. The split keeps a database problem a database problem.

The data model

3.1 Seven tables

The database is small on purpose. Seven tables cover the whole application.

Table	Holds
users	The single account that owns the data.
categories	Optional catalogue of category names and colours.
transactions	One expense or one income entry.
broker_imports	One row per uploaded statement.
investments	A valued position, optionally linked to its import.
bank_accounts	An account, a savings pot or cash.
net_worth_snapshots	Net worth on a given day.

Table 3.1: Tables and what they represent.

There is no table of users' preferences, no audit log and no soft deletes. The application serves one person; features that only pay off with many users would cost complexity and return nothing.

3.2 Money is never a float

Two type aliases are declared once and reused in every model:

```
MONEY = Numeric(14, 2)
QTY = Numeric(20, 8)
```

MONEY covers amounts: fourteen digits with two decimals. QTY covers quantities, which need far more decimals because a crypto position can be a fraction of a coin.

The reason is the classic one. In binary floating point, $0.1 + 0.2$ is not 0.3 . Adding a few hundred expenses that way produces a total that is off by cents, and a total that is off by cents cannot be checked against a bank statement. Python's `Decimal` and SQL's `NUMERIC` do exact decimal arithmetic, so the total either matches or the data is wrong; the arithmetic itself is never in doubt.

The rule in practice

Parsers return `Decimal`. The ORM stores `NUMERIC`. The backup writes decimals as strings, not JSON numbers. Floats appear only in the payloads sent to the charting library, where a rounding error is smaller than a pixel.

3.3 Transactions

```
class Transaction(Base):
    __tablename__ = "transactions"

    id: Mapped[int] = mapped_column(primary_key=True)
    date: Mapped[dt.date] = mapped_column(Date, index=True)
    year: Mapped[int] = mapped_column(Integer, index=True)
    month: Mapped[int] = mapped_column(Integer, index=True)
    category: Mapped[str] = mapped_column(String(120), index=True)
    kind: Mapped[str] = mapped_column(String(10))          # expense | income
    amount: Mapped[Decimal] = mapped_column(MONEY)
    note: Mapped[str | None] = mapped_column(String(255), nullable=True)
```

Two choices in that model deserve an explanation.

3.3.1 Why year and month are stored separately

The date is already there, so storing the year and the month again looks like duplication. It is, and it is deliberate.

Every view in the application groups by month. Extracting a month from a date inside a query means calling a database function, and those functions differ between Postgres and SQLite; worse, an index on the date column usually cannot be used once the value is wrapped in a function call. Two indexed integer columns behave identically on both engines and let the planner use the index.

The cost is that the two representations could disagree. They cannot in practice, because every write goes through a handful of functions that set all three fields together.

3.3.2 Why the category is a string

There is a categories table, but transactions store the category name directly instead of a foreign key. That means a user can type a new category and it simply exists, with no extra step and no dialogue asking to create it first.

The price is that renaming a category later means an update across rows. For a single-user application with a few thousand transactions, that trade is clearly worth it.

3.4 Investments

```
class Investment(Base):
    __tablename__ = "investments"

    broker: Mapped[str] = mapped_column(String(30), index=True)
    asset: Mapped[str] = mapped_column(String(200))
    isin: Mapped[str | None]
    quantity: Mapped[Decimal | None] = mapped_column(QTY, nullable=True)
    invested: Mapped[Decimal] = mapped_column(MONEY, default=Decimal("0"))
    current_value: Mapped[Decimal] = mapped_column(MONEY, default=Decimal("0"))
    withdrawn: Mapped[Decimal] = mapped_column(MONEY, default=Decimal("0"))
    profit: Mapped[Decimal] = mapped_column(MONEY, default=Decimal("0"))
    valued_on: Mapped[dt.date] = mapped_column(Date, index=True)

    # automatic revaluation
    yahoo_symbol: Mapped[str | None]
    current_price: Mapped[Decimal | None] = mapped_column(QTY, nullable=True)
    price_updated_at: Mapped[dt.datetime | None]
    auto_value: Mapped[bool] = mapped_column(Boolean, default=True)
```

```
monthly_contribution: Mapped[Decimal] = mapped_column(MONEY, default=Decimal("0"))
```

A row is a position *as valued on a date*, not a position in general. Every statement import writes a fresh set of rows with today's `valued_on`, and the old rows stay. That gives a free history of the portfolio, and it means an import can never destroy earlier data.

The consequence is that reading "the portfolio" means reading, for each broker, only the rows of its most recent valuation. That query lives in one function, `latest_investments()`, and everything else calls it.

```
def latest_investments(db: Session) -> list[Investment]:
    rows = []
    for broker in db.scalars(select(Investment.broker).distinct()).all():
        max_date = db.scalar(
            select(func.max(Investment.valued_on)).where(Investment.broker == broker)
        )
        rows.extend(db.scalars(select(Investment).where(
            Investment.broker == broker, Investment.valued_on == max_date)).all())
    return rows
```

The loop runs per broker rather than once for the whole table because brokers are imported on different days. Taking a single global maximum date would hide every broker that was not imported most recently.

3.5 Bank accounts and net worth snapshots

Bank accounts are deliberately dumb: a name, a kind, a balance and a timestamp. The user updates the balance when convenient; the application never tries to connect to a bank.

`net_worth_snapshots` holds one row per day with three amounts: investments, banks and the total. It is written by an upsert, so several changes on the same day update the same row.

```
def snapshot_today(db: Session) -> NetWorthSnapshot:
    inv_value, bank_value = _net_worth_parts(db)
    today = date.today()
    snap = db.scalar(select(NetWorthSnapshot).where(NetWorthSnapshot.date == today))
    if snap is None:
        snap = NetWorthSnapshot(date=today)
        db.add(snap)
    snap.investments = Decimal(str(round(inv_value, 2)))
    snap.banks = Decimal(str(round(bank_value, 2)))
    snap.total = snap.investments + snap.banks
    db.commit()
    return snap
```

Why store history instead of computing it

Net worth could be recomputed from current values whenever the chart is drawn. It would also redraw the past every time a price moved, because the only prices available are today's. Storing a snapshot freezes what was true on the day, which is the whole point of an evolution chart.

The snapshot is written after any change that can move net worth: editing an investment, contributing to one, editing an account, or the nightly job. Missing one of those calls does not corrupt anything, it only leaves a gap in the chart.

3.6 Migrations

Schema changes go through Alembic. The environment file reads the URL and the metadata from the application itself, so there is no second copy of the connection settings:

```
config.set_main_option("sqlalchemy.url", settings.sqlalchemy_url)
target_metadata = Base.metadata
...
context.configure(connection=connection, target_metadata=target_metadata,
                  render_as_batch=settings.is_sqlite)
```

`render_as_batch` matters when developing against SQLite, which cannot alter most things in place. In batch mode Alembic creates a new table, copies the data and swaps it, which reproduces on SQLite what Postgres does with a plain `ALTER TABLE`.

The usual cycle is two commands:

```
alembic revision --autogenerate -m "what changed"
alembic upgrade head
```

Autogenerate compares the models with the live database and writes the difference. It is a starting point, not an oracle: it does not see renames, and it does not know how to move data. Every generated file is worth reading before it is applied.

Expenses and income

4.1 The problem with data entry

Recording expenses is not a technical problem, it is a habit problem. Any application that asks for six fields per coffee gets abandoned in a week.

The user of this application already had a habit: writing the month's expenses in the notes app of a phone, as a list with bullets, sometimes with small sums written out because three purchases went into the same category.

```
June
- Expense: Groceries: 15-1+3
- Income: Salary: 1200
- Food: 8.53+1+26.25
- Transport: (10+5)*2
```

Rather than replace that habit, the application reads it. The user pastes the text, and the parser turns it into rows.

4.2 Parsing a pasted block

The parser lives in `app/services/expenses_parse.py` and works in three steps.

1. **Find the month.** If the first non-empty line is a month name and contains no colon, it is the title and it sets the month.
2. **Clean each line.** A regular expression removes leading bullets: -, *, ·, and a few others that phones insert.
3. **Split by colon.** The last field is the arithmetic expression, everything before it is a label. If the first label is a kind (*Expense* or *Income*) it sets the kind, otherwise it is part of the category.

Lines that do not fit come back in an ignored list instead of being dropped in silence. The preview shows them, so a typo is visible rather than mysterious.

```
@dataclass
class ParsedItem:
    kind: str          # expense | income
    category: str
    amount: Decimal
    note: str | None = None

@dataclass
class ParsedExpenses:
    title: str
    month: int | None
    items: list[ParsedItem]
    ignored: list[str] = field(default_factory=list)
```

4.3 Evaluating arithmetic without eval

A line can contain $15-1+3$ or $(10+5)*2$. The obvious way to evaluate that is `eval()`, and it is the wrong way: `eval` on text that came from outside runs whatever that text says.

The parser uses Python's own `ast` module instead. The expression is parsed into a syntax tree, and the tree is walked by a function that accepts only numbers, unary signs, and six binary operators.

```
_BINOPS = {ast.Add: operator.add, ast.Sub: operator.sub,
            ast.Mult: operator.mul, ast.Div: operator.truediv,
            ast.FloorDiv: operator.floordiv, ast.Mod: operator.mod}

def _eval_node(node: ast.AST) -> float:
    if isinstance(node, ast.Constant):
        if isinstance(node.value, bool) or not isinstance(node.value, (int, float)):
            raise ExprError("numbers only")
        return float(node.value)
    if isinstance(node, ast.BinOp):
        op = _BINOPS.get(type(node.op))
        if op is None:
            raise ExprError("operator not allowed")    # ** lands here
        left, right = _eval_node(node.left), _eval_node(node.right)
        if op in (operator.truediv, operator.floordiv, operator.mod) and right == 0:
            raise ExprError("division by zero")
        return op(left, right)
    if isinstance(node, ast.UnaryOp) and isinstance(node.op, (ast.UAdd, ast.USub)):
        val = _eval_node(node.operand)
        return val if isinstance(node.op, ast.UAdd) else -val
    raise ExprError("expression not allowed")
```

Anything the walker does not recognise raises. A name, a function call, an attribute or a subscript never reaches an evaluation branch, because there is no branch for them.

Why exponentiation is blocked

`**` is arithmetic and would fit the theme, but `2**9999999` is a denial of service in eleven characters: it is a legal expression that takes a long time and a lot of memory. It is left out of the operator table, and the input is capped at 120 characters as a second line of defence.

Two more guards run before parsing: the expression must match a character allowlist (digits, spaces, `.`, `+`, `-`, `*`, `/`, `%`, `(`, `)`), and a comma is normalised to a dot so that both decimal conventions work.

4.4 The preview step

Parsing produces a preview page, not database rows. The preview shows each detected line with an editable category, an editable amount, a kind selector and a checkbox, plus the list of ignored lines.

Saving offers two buttons, and the difference between them matters:

- **Append** adds the rows to whatever the month already contains.
- **Replace** deletes every row of that month first.

Replace exists because the realistic workflow is pasting a month, noticing a mistake, fixing the text and pasting again. Without it, the second paste would double the month. It asks for confirmation in the browser, since it destroys data.

```
if mode == "replace":
    db.execute(sql_delete(Transaction).where(
        Transaction.year == year, Transaction.month == month))
```

Note the two conditions: the delete is scoped to one month of one year, never to the table. A test covers exactly that, by seeding a row in a different month and asserting it survives.

4.5 Reading amounts from a form

Users type amounts in whatever convention they are used to. The form parser accepts all four common forms:

```
def _parse_amount(raw: str) -> Decimal:
    s = raw.strip().replace(" ", "")
    if "," in s and "." in s:
        # the last separator is the decimal one; the other groups thousands
        if s.rfind(",") > s.rfind("."):
            s = s.replace(".", "").replace(",", ".")
        else:
            s = s.replace(",", "")
    else:
        s = s.replace(",", ".")
    try:
        return Decimal(s).quantize(Decimal("0.01"))
    except (InvalidOperation, ValueError):
        return Decimal("0.00")
```

The rule is simple and works for both conventions: whichever separator appears last is the decimal separator. 1.234,56 and 1,234.56 both become 1234.56.

An unparseable value becomes zero rather than raising. A form that rejects the whole submission because of one bad field is more annoying than a zero the user can see and correct.

4.6 The two views

The monthly view lists the entries of one month with totals for income, expenses and the balance, plus two doughnut charts by category. The yearly view groups by month and shows a bar chart of income against expenses, plus expenses by category for the whole year.

Both read through the same aggregation helpers used by the reports and the bot, so the number on the page and the number in the PDF come from the same query.

4.7 Investing is spending, and also not

One category needed special treatment. Money moved into an investment leaves the current account like an expense, but it is not consumption: it is still part of net worth. Counting it as a plain expense makes a good month look terrible.

The application keeps it simple. Such entries are still expenses, but categories named *investment*, *investments* or *investing* are also summed separately, and every summary adds a line: *of which X was invested*.

```
INVESTMENT_CATEGORIES = {"investment", "investments", "investing"}

def invested_in_period(db, year, month) -> float:
    cond = [Transaction.year == year, Transaction.kind == "expense",
```



```
func.lower(Transaction.category).in_(tuple(INVESTMENT_CATEGORIES))]  
if month:  
    cond.append(Transaction.month == month)  
return _f(db.scalar(select(func.sum(Transaction.amount)).where(*cond)))
```

The comparison is done in lower case inside the query, so the user can capitalise the category however they like.

Investments

5.1 Three brokers, three formats

The application imports statements from three brokers, and none of them export the same thing.

Broker	Format	What the file actually contains
MyInvestor	CSV	Cash movements of the account. Fund purchases appear as negative amounts, redemptions as positive ones.
Kraken	CSV	Individual trades: pair, side, cost and volume.
Trade Republic	PDF	A statement in prose, one transaction per paragraph.

Table 5.1: What each export really is.

5.2 One shape for all of them

Each parser returns the same two dataclasses, so nothing downstream needs to know which broker produced the data.

```
@dataclass
class Position:
    asset: str
    isin: str | None = None
    quantity: Decimal | None = None
    invested: Decimal = Decimal("0")
    withdrawn: Decimal = Decimal("0")
    realised_pnl: Decimal = Decimal("0")
    looks_sold: bool = False
    meta: dict = field(default_factory=dict)

@dataclass
class ParseResult:
    broker: str
    positions: list[Position]
    warnings: list[str] = field(default_factory=list)
```

The warnings list is the error channel. A parser never raises on a bad file; it returns an empty position list and an explanation. The upload page then shows the explanation and the request completes normally. An exception would produce a 500 page that tells the user nothing.

```
def parse(broker: str, content: bytes) -> ParseResult:
    module_path = PARSER_MODULES.get(broker)
    if module_path is None:
        return ParseResult(broker, [], [f"Unknown broker: {broker}"])
```

```
return import_module(module_path).parse(content)
```

The dispatcher imports the parser module by name, at call time. That is not laziness for its own sake: the parsers pull in pandas and pdfplumber, which take seconds to import, and no user should pay that cost on a request that has nothing to do with investments.

5.3 Reading each format

5.3.1 MyInvestor

The export is a cash statement, so the first job is telling fund movements apart from transfers, interest and tax withholdings. Two rules do it: a list of words that mark a cash movement, and the observation that fund names are printed in upper case.

```
def is_fund_row(concept: str) -> bool:
    c = str(concept).strip()
    if any(w in c for w in SKIP_WORDS):
        return False
    if re.search(r"[a-z]", c):      # fund names are upper case
        return False
    return True
```

The remaining rows are grouped by fund. Negative amounts add up to invested, positive ones to withdrawn. When the two nearly match, the position is flagged as probably sold, and the preview shows the flag already ticked.

5.3.2 Kraken

The export lists trades, so a position has to be reconstructed. For each pair, the parser subtracts sells from buys, in both money and volume.

```
net_invested = buys["cost"].sum() - sells["cost"].sum()
volume = buys["vol"].sum() - sells["vol"].sum()
if volume > 1e-7 or abs(net_invested) > 0.01:
    positions.append(Position(asset=str(pair), ...))
```

The threshold on the volume avoids keeping positions that are only rounding dust after everything was sold, while the money condition keeps a pair that is closed but still shows a profit or a loss.

5.3.3 Trade Republic

The hardest one. The statement is a PDF written for humans, so the parser flattens every page to text, splits it by transaction date, and matches each block with regular expressions.

```
ISIN_RE = r"[A-Z]{2}[A-Z0-9]{9}[0-9]"

std = re.search(rf"(Buy|Sell|Savings).*?({ISIN_RE}).*?quantity:\s*([\d\.\.]+)",
                desc, re.IGNORECASE)
```

The extraction is fragile by nature; a change in the statement layout will break it. Three details make it less fragile than it could be. Header and footer lines are removed first. Amounts glued to quantities by the PDF text extraction are split again with targeted substitutions. And

the ISIN is matched by its actual pattern, two country letters, nine alphanumerics and a check digit, rather than by a hardcoded list of the products the author happens to own.

Generic beats specific

The original private version had a special case for one particular product, with its ISIN written in the source. The public version replaced it with a second, looser regular expression that accepts any ISIN followed by a quantity. The parser became shorter and covers more cases.

Trade Republic also allows partial sells, so the parser keeps a running average cost per ISIN and realises the profit of each sell:

```
elif "sell" in op:
    if qty > 1e-6:
        avg = cost / qty
        sold_cost = avg * q
        pnl += total - sold_cost
        qty -= q
        cost -= sold_cost
```

That realised profit travels in `Position.realised_pnl` and is added back later, because otherwise a profit that was already taken would disappear from the totals.

5.4 Profit depends on the broker

Given the three formats above, a single profit formula would be wrong for at least two of them. The calculation branches instead:

```
def total_profit(broker, invested, withdrawn, current_value,
                 realised_pnl=ZERO, sold=False) -> Decimal:
    if broker == "myinvestor":
        val_calc = ZERO if (sold and withdrawn > invested * HALF) else current_value
        return ((val_calc + withdrawn) - invested).quantize(Decimal("0.01"))
    if broker == "kraken":
        return (current_value - invested).quantize(Decimal("0.01"))
    if broker == "traderpublic":
        return ((current_value - invested) + realised_pnl).quantize(Decimal("0.01"))
    return ((current_value + withdrawn) - invested + realised_pnl).quantize(Decimal("0.01"))
```

Each branch answers a different question:

- **MyInvestor** reports redemptions, so money taken out counts. When a fund was sold and most of the money already came back, the current value is ignored, otherwise the same money would be counted twice.
- **Kraken** nets buys against sells during parsing, so the profit is simply what the position is worth now minus what is still committed.
- **Trade Republic** already realised part of the profit on partial sells, so it is added to the unrealised part.

Every branch is covered by a test, including the double counting case, which is the one that is easy to get wrong and impossible to notice by eye.

5.5 Valuation: manual first

A position needs a current value. Fetching it automatically works well for listed products and badly for funds, whose share classes are hard to identify and whose prices are often stale or missing.

The application therefore makes automatic pricing **opt-in**. When adding a position the user ticks a box, or pastes a symbol; otherwise the position keeps the value that was typed and only the profit is recomputed.

```
def refresh_position(inv: Investment) -> bool:
    if not inv.auto_value:
        return False
    symbol = inv.yahoo_symbol or (
        crypto_symbol(inv.asset) if inv.broker == "kraken" else None)
    if not symbol or inv.quantity is None:
        return False
    price = fetch_price(symbol)
    if price is None:
        return False
    inv.current_price = price
    inv.price_updated_at = datetime.now(timezone.utc)
    inv.current_value = (Decimal(inv.quantity) * price).quantize(Decimal("0.01"))
    inv.profit = total_profit(...)
    return True
```

The function returns a boolean, and every guard returns False instead of raising. A failed price lookup is normal: markets close, tickers change, networks fail. The position simply keeps its previous value, which is stale but correct, rather than being wiped to zero.

Silent failure is right here, wrong elsewhere

Swallowing an error is usually a bad habit. It is right here because the alternative, an exception during a nightly job, would leave the whole portfolio unrefreshed because of one delisted ticker. Compare with the backup in chapter 9, where a failure must be loud.

The batch version collects errors per position and keeps going:

```
for inv in positions:
    try:
        if refresh_position(inv):
            updated += 1
    except Exception as exc:
        # one bad symbol must not abort the batch
        errors.append(f"{inv.asset}: {exc}")
```

5.6 From an ISIN to a ticker

Users know the ISIN of a product because it is printed on every statement. They rarely know its Yahoo Finance symbol, which also depends on the exchange. The application resolves one into the other in two steps.

1. **Yahoo search.** Every suggested symbol is probed with a price request, preferring symbols quoted in euros. This is what resolves funds, whose symbols look like 0P0001CLDK.F.
2. **OpenFIGI.** A free mapping API that returns a ticker and an exchange code, which are combined into a Yahoo symbol through a suffix table. This works well for listed shares and ETFs.

```

EXCH_TO_YAHOO_SUFFIX = {
    "GR": ".DE", "GY": ".DE", "GF": ".DE", # Germany
    "SM": ".MC", # Spain
    "FP": ".PA", "NA": ".AS", "BB": ".BR", # Euronext
    "LN": ".L", "IM": ".MI", "SW": ".SW",
    "US": "", "UN": "", "UW": "", "UQ": "", # United States, no suffix
}

```

When a product is listed on several exchanges, European venues are preferred, so the price comes back in the same currency as the rest of the portfolio.

Results are cached in a module-level dictionary, including the negative ones. An ISIN that resolves to nothing resolves to nothing all day, and there is no reason to ask twice.

5.7 Contributions

Regular investing means adding money to the same position every month. The model keeps a single invested figure, the accumulated cost, which grows with every contribution. Profit is then always the value minus what was actually put in.

```

def contribute(db, inv, amount, units=ZERO) -> None:
    inv.invested = Decimal(inv.invested or 0) + amount
    if units:
        inv.quantity = Decimal(inv.quantity or 0) + Decimal(units)

    if not (inv.auto_value and refresh_position(inv)):
        inv.current_value = Decimal(inv.current_value or 0) + amount
        inv.profit = total_profit(...)

```

The conditional is the interesting part. For an automatically valued position, the market price already reflects the new units, so adding the contributed amount to the value would count it twice. For a manual position there is no price, so the contributed money enters the value directly and the user adjusts later.

A position can also carry a `monthly_contribution`. A scheduled job on the first of each month applies all of them, logging and skipping any that fail, so one broken position does not stop the rest.

Net worth and aggregation

6.1 What was missing

An early version of the application tracked investments and called their value "net worth". That was wrong in an obvious way: it ignored the money sitting in bank accounts, which for most people is a large part of what they have.

Adding accounts was easy. Deciding how they participate in the rest of the application was the real work.

6.2 Accounts

An account is a name, a kind (account, savings or cash), a balance and a timestamp. There is no bank integration, and that is a decision rather than a limitation: connecting to a bank means credentials, aggregation providers and a recurring cost, for data the user can update in ten seconds a month.

Every write to an account ends the same way:

```
db.commit()
snapshot_today(db)
```

So the evolution chart follows real changes instead of only moving when a statement is imported.

6.3 The aggregation module

`app/services/aggregation.py` is the only place that computes totals. The web pages, the PDF reports and the Telegram bot all read from it, which is what guarantees that the same number appears everywhere.

Its main function builds the whole dashboard payload in one pass:

```
def build_dashboard(db, year=None, month=None) -> dict:
    invs = latest_investments(db)
    inv_value = sum(_f(i.current_value) for i in invs)
    inv_invested = sum(_f(i.invested) for i in invs)
    inv_profit = sum(_f(i.profit) for i in invs)
    return_pct = (inv_profit / inv_invested * 100) if inv_invested else 0.0

    banks = db.scalars(select(BankAccount).order_by(BankAccount.balance.desc())).all()
    bank_value = sum(_f(b.balance) for b in banks)
    net_worth = inv_value + bank_value
    ...
    return {"year": year, "month": month, "kpis": {...}, "charts": {...}}
```

Note the guard on the division. An empty portfolio has zero invested, and a return of zero is the honest answer; raising or returning infinity would be worse.

6.4 Month ranges without special cases

Several features ask for a range of months: a report from October 2025 to March 2026, a cashflow chart, a net figure for any period. Written naively, those queries fill up with conditions about years and boundaries.

The module converts a year and a month into a single integer, the month ordinal:

```
def _month_ordinals(y1, m1, y2, m2) -> tuple[int, int]:
    start, end = y1 * 12 + m1, y2 * 12 + m2
    return (end, start) if start > end else (start, end)

_ORDINAL = Transaction.year * 12 + Transaction.month
```

A range then becomes a plain comparison, `_ORDINAL >= start` and `_ORDINAL <= end`, with no year logic at all. The same expression is built once by SQLAlchemy and reused by every range query.

Reversed inputs are normalised in the same helper, so a user who types the end month first gets the range they meant instead of an empty result.

Turning an ordinal back into a year and a month is one line:

```
for o in range(start, end + 1):
    y, m0 = divmod(o - 1, 12)
    months.append((y, m0 + 1))
```

The general lesson

When a range keeps producing boundary bugs, it is usually the representation that is wrong, not the conditions. Mapping the values onto a single ordered number makes the boundaries disappear.

6.5 The dashboard

The dashboard answers two questions on one screen: how much does the user have, and where did this month go.

Four indicators sit at the top: net worth (with the split between investments and banks, plus profit and return), income of the month, expenses of the month, and savings of the month with the invested part noted underneath.

Four charts follow:

- income against expenses per month of the year, as bars;
- expenses by category for the selected month, as a doughnut;
- net worth over time, as a line, from the snapshot table;
- the split of net worth by broker and account, as a doughnut.

The net worth series has a fallback. When there are no snapshots yet, because the application was just installed, it groups investments by valuation date instead, so the chart is not empty on the first day.

```
if snaps:
    evo_labels = [d.strftime("%d/%m/%y") for d, _ in snaps]
    evo_data = [round(_f(s), 2) for _, s in snaps]
else:
    evo = db.execute(select(Investment.valued_on, func.sum(Investment.current_value))
```



```
.group_by(Investment.valued_on)
.order_by(Investment.valued_on)).all()
```

6.6 An empty database is a valid state

Every aggregation function is written so that a fresh installation renders correctly. Sums return zero instead of None, thanks to a small helper:

```
def _f(x) -> float:
    return float(x or 0)
```

The list of years falls back to the current year when there are no transactions, so the year selector is never empty. And the payload carries a `has_data` flag, which the template uses to show a short message pointing at the two places where a new user should start.

It is a small thing, and it is the difference between an application that looks broken on day one and one that looks ready.

Reports

7.1 Why reports at all

A dashboard answers questions while the user is looking at it. A report is something that can be filed, sent, or read a year later. The application produces both formats people actually use: Excel for anyone who wants to poke at the numbers, and PDF for anything that gets read rather than edited.

7.2 Excel

The Excel export is built with `openpyxl`. All the styling lives in one function, so every sheet looks the same:

```
def _write_sheet(ws, headers: list[str], rows: list[list]) -> None:
    for c, h in enumerate(headers, 1):
        cell = ws.cell(row=1, column=c, value=h)
        cell.fill = _HEADER_FILL
        cell.font = _HEADER_FONT
    for r, row in enumerate(rows, 2):
        for c, val in enumerate(row, 1):
            cell = ws.cell(row=r, column=c, value=val)
            if r % 2 == 0:
                cell.fill = _ALT_FILL
            if isinstance(val, (int, float, Decimal)) and c > 1:
                cell.number_format = _money_format()
    for col in ws.columns:
        width = max((len(str(c.value)) for c in col if c.value is not None), default=8)
        ws.column_dimensions[get_column_letter(col[0].column)].width = width + 4
```

Two details matter. Amounts are written as numbers with a currency number format, not as pre-formatted strings, so the spreadsheet can still sum them. And the column width is computed from the longest value, which removes the usual first action of every reader, widening the columns by hand.

7.3 PDF

The PDF reports use `reportlab` for the document and `matplotlib` for the charts, which are rendered to PNG in memory and embedded.

```
def _save(fig) -> bytes:
    buf = io.BytesIO()
    fig.savefig(buf, format="png", dpi=130, bbox_inches="tight", facecolor="white")
    plt.close(fig)
    buf.seek(0)
    return buf.getvalue()
```

Nothing touches the filesystem. The chart exists as bytes, goes into the document, and disappears. On a hosting platform with a read-only or temporary filesystem that is the only

approach that works, and it avoids leaving files behind after a failed request.

The Agg backend is selected explicitly before importing pyplot, because the default backend expects a display that a server does not have.

Four reports are available: monthly, yearly, any range of months, and a detailed one that breaks down net worth position by position and account by account.

7.4 A bug worth keeping

The first version of the doughnut chart crashed on a real month. The cause was a category with a negative total, produced by a refund recorded as a negative expense.

`matplotlib.pie` raises on slices that are zero or negative, which is reasonable: a pie chart of a negative share has no meaning. The fix filters the data for the chart while leaving the tables untouched:

```
pairs = [(l, float(v)) for l, v in zip(labels, data) if v and float(v) > 0]
if not pairs:
    ax.text(0.5, 0.5, "No data", ha="center", va="center")
    ax.axis("off")
    return _save(fig)
```

What the fix teaches

The chart hides what it cannot draw; the report still shows it in the table below. Dropping the row entirely would have been a data loss disguised as a rendering fix. A regression test now generates a report from a month containing a negative category.

7.5 Percentages that do not lie

Formatting percentages sounds trivial until a real number rounds badly. A return of 0.4% displayed with zero decimals becomes 0%, which reads as "this investment did nothing" when in fact it did something small.

```
def pct(value, decimals: int = 0, zero_decimals: int = 3) -> str:
    v = float(value or 0)
    d = zero_decimals if (v != 0 and round(v, decimals) == 0) else decimals
    return f"{v:.{d}f}%"
```

A real zero still prints as 0%. A value that only rounds to zero gets extra precision instead. The rule is one line and it removes an entire class of misleading output.

7.6 Lazy imports, again

The reports module pulls in the four heaviest dependencies in the project. They are imported inside each handler, never at module level:

```
@router.get("/pdf/monthly")
def pdf_monthly(year: int, month: int, ...):
    from app.services.reports import generate
    return _pdf(generate.monthly_pdf(db, year, month), f"Report_{MONTHS[month]}_{year}.pdf")
```

The cost of an import inside a function is one dictionary lookup after the first call, since Python caches modules in `sys.modules`. The benefit is several seconds off every cold start, which chapter 9 shows to be the difference between a working bot and a silent one.

A test enforces the rule so it cannot rot. It runs a clean subprocess, imports `app.main`, and asserts that none of the heavy modules ended up loaded.

The Telegram bot

8.1 Why a bot

The web application works on a phone, but opening a browser, logging in and filling a form is still friction, and friction is what kills the habit of recording expenses. A chat is already open on the phone, so the bot removes the last excuse: the user types the same lines they used to write in a notes app, and they land in the database.

8.2 No extra dependency

The bot talks to the Telegram Bot API over plain HTTP with `httpx`, which the project already uses. There is no bot framework.

```
API = "https://api.telegram.org/bot{token}/{method}"

async def send_message(chat_id, text, reply_markup=None) -> None:
    payload = {"chat_id": chat_id, "text": text, "parse_mode": "HTML",
              "disable_web_page_preview": True}
    if reply_markup is not None:
        payload["reply_markup"] = reply_markup
    async with httpx.AsyncClient(timeout=TIMEOUT) as c:
        await c.post(_url("sendMessage"), json=payload)
```

The API is small: send a message, edit a message, answer a callback, send a document. Four functions cover everything the application does, and adding a framework to wrap four HTTP calls would have added a dependency, a second event loop model and a second way to configure things.

8.3 Webhook instead of polling

There are two ways to receive updates. Polling asks Telegram for new messages in a loop, which needs a process that is always awake. A webhook lets Telegram POST each update to a URL.

The application uses a webhook, because on a free host nothing is guaranteed to stay awake, and an incoming request is exactly what wakes it up.

```
async def setup_webhook() -> None:
    if not settings.public_base_url:
        return
    url = settings.public_base_url.rstrip("/") + "/tg/webhook"
    payload = {"url": url, "drop_pending_updates": True,
              "allowed_updates": ["message", "callback_query"]}
    if settings.telegram_webhook_secret:
        payload["secret_token"] = settings.telegram_webhook_secret
    ...
```

`drop_pending_updates` avoids replaying a backlog after a redeploy, and `allowed_updates` keeps Telegram from sending update types the application ignores.

8.4 Two locks on the door

The webhook URL is public, so anybody can POST to it. Two independent checks protect it.

First, the secret token. It is registered with the webhook, and Telegram sends it back in a header on every update. The router compares it in constant time:

```
secret = settings.telegram_webhook_secret
if secret and not secrets.compare_digest(x_telegram_bot_api_secret_token or "", secret):
    return Response(status_code=403)
```

Second, the chat allowlist. Even a genuine Telegram update is only served if it comes from an allowed chat, because anybody who finds the bot name can message it.

```
def is_allowed(chat_id) -> bool:
    allow = settings.allowed_chat_ids
    return (not allow) or (str(chat_id) in allow)
```

An empty allowlist means setup mode: the bot replies with the chat id so the owner can copy it into the configuration. Once the allowlist has a value, anything else is ignored in silence, with a line in the log. Answering an intruder only confirms that the bot exists.

Never answer 5xx to Telegram

The webhook returns 200 even when handling the update failed. Telegram retries on server errors, and a bug that raises on one message would turn into an endless retry of the same message. The error is logged; the update is acknowledged.

8.5 Confirm before writing

Free text is read by the same parser the web import uses. The result is not saved: it is shown as a preview with two buttons.

```
token = secrets.token_urlsafe(6)
_PENDING[token] = (chat_id, parsed)
while len(_PENDING) > _MAX_PENDING:
    _PENDING.popitem(last=False)          # drop the oldest
...
await send_message(chat_id, body, reply_markup=_kb([
    [("Confirm", f"c:ok:{token}")], ("Cancel", f"c:no:{token}")]))
```

Pressing a button sends a callback with that token, and only then are the rows inserted. The message is edited in place with the result, so the chat shows what happened instead of growing a new message.

Pending previews live in a capped `OrderedDict` in memory. That is a deliberate choice with two consequences: a restart loses them, and an old preview is evicted after fifty newer ones. Both are safe, because losing a pending preview means nothing was written. Storing them in the database would have meant a table, a cleanup job and a way to expire rows, to protect data that does not exist yet.

8.6 Periods and ranges

Several commands accept a period. The parser handles a month, a year, or a range of either, and returns a label for the report title.

```
def _parse_range(arg: str) -> tuple[int, int, int, int, str]:
    toks = [t for t in (arg or "").split() if t.lower() not in _CONNECTORS]
    if not toks:
        y, m = _parse_period("")
        return y, m, y, m, f"{MONTHS[m]} {y}"
    if len(toks) == 1:
        y, m = _parse_period(toks[0])
        if m:
            return y, m, y, m, f"{MONTHS[m]} {y}"
        return y, 1, y, 12, str(y)  # a year alone means the whole year
    ya, ma = _parse_period(toks[0])
    yb, mb = _parse_period(toks[-1])
    return ya, (ma or 1), yb, (mb or 12), f"{MONTHS[ma or 1][:3]} {ya} - ..."
```

Connector words such as *to* and *until* are dropped, so `/report 01/2026 to 03/2026` and `/report 01/2026 03/2026` both work. A year without a month opens at January and closes at December, which is what a person means when they type a year.

The report command then picks the right generator: a single month gives a monthly report, a full year gives a yearly one, anything else gives a range report.

8.7 Commands

Command	Answer
<code>/menu</code>	Button menu, which is faster than typing on a phone.
<code>/summary [mm/yyyy]</code>	Income, expenses, savings and net worth.
<code>/net [period]</code>	Income, expenses and the net figure, any range.
<code>/networth</code>	Investments and accounts, with profit and return.
<code>/banks, /bank <name></code> <code><balance></code>	Read or update balances.
<code>/expenses [mm/yyyy]</code>	Expenses by category.
<code>/cashflow [yyyy]</code>	Income against expenses, month by month.
<code>/savings [mm/yyyy]</code>	Savings of the period.
<code>/report [period]</code>	PDF with charts.
<code>/detailed [period]</code>	Full PDF: net worth, categories, cashflow.

Table 8.1: The bot command set.

The `/bank` command is the only one that writes without confirmation, since it overwrites a single number the user just typed and the reply shows the new total.

8.8 Synchronous and asynchronous sending

The module has two ways to send. The async functions serve the webhook. A pair of synchronous ones serve the scheduler, which runs in a background thread with no event loop.

They differ in how they treat failure, and the difference is intentional:

```
def send_document_sync(chat_id, data, filename, caption="", mime="application/pdf"):
    r = httpx.post(_url("sendDocument"), ...)
    r.raise_for_status()  # a failed backup must be loud
```

```
def send_message_sync(chat_id, text) -> None:
    try:
        httpx.post(_url("sendMessage"), ...)
    except Exception as exc:
        log.warning("send_message_sync failed: %s", exc)    # a lost reminder is fine
```

A month-end reminder that does not arrive costs nothing. A backup that fails silently is worse than having no backup at all, because it produces false confidence. So one swallows the error and the other propagates it up to the task endpoint, which turns it into a `500` that the cron shows in red.

Keeping it running

9.1 The failure that started this chapter

The application was deployed on a free plan and worked. Some days later the bot stopped answering. Nothing had been changed, no error appeared in the logs, and opening the web page worked normally after a short wait.

The cause was a combination of two behaviours that are documented, free, and easy to overlook:

- the hosting platform **suspends** the application after about fifteen minutes without traffic;
- the managed database **pauses** the project after several days without queries.

Neither is a bug. Together they break the bot in a way that is invisible: when a message arrives, Telegram POSTs to a process that is still starting. Telegram waits a few seconds, gives up, and drops the update. By the time the application is ready, there is nothing left to answer. The user sees a bot that ignores them, and the logs show a clean startup.

The lesson

A deployment is not finished when the application answers. It is finished when it still answers a week later without anybody touching it. That last part is a feature, and it has to be designed like one.

9.2 Three pieces

Piece	What it does	Where it runs
Uptime monitor	GET /health/db every five minutes	An external monitor
Task cron	POST /tasks/* on a schedule	An external cron
Backup	POST /tasks/backup weekly	The same cron

Table 9.1: What keeps the application alive on a free plan.

The monitor solves both problems at once. Each request keeps the application from being suspended, and because the endpoint queries the database, it also keeps the database from being paused. A single ping every five minutes replaces both problems with none.

9.3 Why the health endpoints differ

```
@app.get("/health")
def health():
    """Liveness: says the process answers. It does not touch the database."""
    return {"status": "ok", "db": "sqlite" if settings.is_sqlite else "postgres"}
```

```
@app.get("/health/db")
def health_db():
    """Readiness: a real SELECT 1, and 503 when the database is silent."""
    db = SessionLocal()
    try:
        db.execute(text("SELECT 1"))
    except Exception as exc:
        return JSONResponse({"status": "error", "db": "unreachable",
                             "detail": str(exc)[:200]}, status_code=503)
    finally:
        db.close()
    return {"status": "ok", ...}
```

The platform healthcheck points at `/health`, which never fails because of the database. If it did, a five minute database outage would make the platform consider the application broken and remove the deployment, so a temporary problem would become a permanent one.

The external monitor points at `/health/db`, which fails loudly on purpose. A paused database with a process still running is exactly the situation that must produce an alert.

Four tests pin this behaviour, including one that patches the session factory to raise and asserts that `/health` still answers 200.

9.4 Scheduled jobs, twice

The same four jobs can run in two different ways.

On an always-on machine, an in-process scheduler is enough:

```
sched.add_job(_refresh_prices, CronTrigger(hour=3, minute=0), id="prices")
sched.add_job(_contributions, CronTrigger(day=1, hour=6, minute=0), id="contributions")
sched.add_job(_snapshot, CronTrigger(hour=23, minute=0), id="snapshot")
if settings.telegram_enabled:
    sched.add_job(_reminder, CronTrigger(day="last", hour=20, minute=0), id="reminder")
```

On a host that sleeps it is useless: a scheduler inside a suspended process does not fire. The same jobs are therefore exposed as HTTP endpoints that an external cron can call.

```
JOBS = {"prices", "contributions", "snapshot", "reminder", "backup"}

@router.post("/{job}")
def run_task(job: str, x_tasks_token: str | None = Header(default=None),
             token: str | None = Query(default=None), db: Session = Depends(get_session)):
    if not _authorised(x_tasks_token, token):
        return JSONResponse({"ok": False, "error": "not authorised"}, status_code=403)
    if job not in JOBS:
        return JSONResponse({"ok": False, "error": "unknown job"}, status_code=404)
    ...
```

A single environment variable, `USE_SCHEDULER`, picks the mode. The job functions are shared, so the two paths cannot drift apart.

9.5 Two ways to send the token

The endpoints accept the token either in a header or in the query string. That looks like sloppy design, and it is the result of a debugging session.

The header is the better option: it keeps the secret out of URLs, out of browser history and out of proxy logs. It was the only option in the first version.

Then the cron service refused to work. Every job returned 403, with no further detail, because a rejected request is all the endpoint is willing to say. The cause was that the free cron service does not forward custom headers to the target. The header was simply never arriving.

```
def _authorised(header_token, query_token) -> bool:
    expected = settings.tasks_token
    if not expected:                # no token configured: endpoints disabled
        return False
    submitted = header_token or query_token or ""
    return secrets.compare_digest(submitted, expected)
```

An empty configured token disables the endpoints entirely, rather than allowing everything, which is the direction a mistake here should fail in. The comparison uses `compare_digest` so that a wrong token cannot be discovered by timing the response.

Two more traps in the same area

The cron URL must use `https://`. With `http://` the platform answers a redirect, and a redirected POST loses its body, so the job appears to run and does nothing.

And a job that fails must return a non-2xx status, or the cron dashboard shows green for a task that never worked. That is why the backup endpoint answers 500 when delivery failed.

9.6 Backups

A free managed database can be paused and, eventually, deleted. The backup therefore has one rule: the copy must not live inside the thing it protects.

The dump is written by walking the SQLAlchemy metadata, so a new table joins the backup without anybody editing the backup code:

```
EXCLUDED_COLUMNS = {"users": {"password_hash"}}

def export_data(db: Session) -> dict:
    tables = {}
    for name, table in Base.metadata.tables.items():
        excluded = EXCLUDED_COLUMNS.get(name, set())
        columns = [c for c in table.columns if c.name not in excluded]
        rows = db.execute(select(*columns)).all()
        tables[name] = [{"c.name": _jsonable(v) for c, v in zip(columns, row)} for row in rows]
    return {"generated": ..., "engine": ..., "tables": tables}
```

Three decisions are worth naming.

JSON, not pg_dump. The container image is `python:3.12-slim` and carries no Postgres client. Dumping through SQLAlchemy adds no dependency, produces the same file whether the source is SQLite or Postgres, and can be read without any tooling.

Decimals as strings. `_jsonable` converts Decimal to a string, never to a JSON number, because JSON numbers are floats and the whole project exists to avoid that.

The password hash never leaves. It is excluded by name, and a test asserts that no bcrypt prefix appears anywhere in the serialised output. The user is recreated after a restore with the seed script, so the hash has no value in the backup and every reason not to travel through a messaging API.

The file is gzipped and sent to the owner's chat, which puts it outside the provider. Restoring is the mirror image, with one safety rule:

```
python -m scripts.backup restore copy.json.gz                # dry run, prints only
```

```
python -m scripts.backup restore copy.json.gz --write # deletes and inserts
```

Restore deletes the contents of every table present in the file, so the destructive path requires an explicit flag. Tables are deleted in reverse dependency order and inserted in forward order, using `Base.metadata.sorted_tables`, so foreign keys hold at every step.

Security

10.1 The threat model

Being honest about the threat model keeps the work proportionate. This application is a single-user site, reachable from the public internet, holding financial history but no card numbers, no bank credentials and no ability to move money.

What it must resist:

- somebody guessing the password;
- somebody reading or forging a session;
- a page on another site making the browser act on the user's behalf;
- a malicious file or a malicious pasted line;
- secrets leaking into the repository.

What it does not try to solve: a compromised machine, a hostile hosting provider, or an attacker who already has the database.

10.2 Passwords

The password is hashed with bcrypt:

```
def hash_password(plain: str) -> str:
    return bcrypt.hashpw(plain.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

def verify_password(plain: str, hashed: str) -> bool:
    try:
        return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("utf-8"))
    except (ValueError, TypeError):
        return False
```

bcrypt is deliberately slow and salts every hash, which is what makes a stolen database expensive to attack. A general-purpose hash such as SHA-256 is fast, and speed is exactly the property an attacker wants.

checkpw compares in constant time, so the verification does not leak information through timing. The except catches malformed stored values and answers "no" rather than raising, which turns a corrupt row into a failed login instead of a 500 page.

10.3 Sessions

The session is a signed cookie handled by Starlette's session middleware. It holds one thing: the user id.

```
app.add_middleware(
    SessionMiddleware,
    secret_key=settings.secret_key,
    https_only=settings.cookie_secure,
    same_site="lax",
```

```

    session_cookie="fm_session",
    max_age=60 * 60 * 12,      # session expires after 12 hours
)

```

The cookie is signed, not encrypted, so the contents are readable by the user but cannot be changed without the key. A user id is not a secret, so that is fine.

`same_site="lax"` stops the cookie from being attached to cross-site requests, which removes most of the CSRF surface before the token below is even considered. `https_only` is tied to configuration so that local development over plain HTTP still works, while production sets it to true.

And, as chapter 2 showed, production refuses to start with the default signing key. A default key published in a public repository would let anyone mint a valid session.

10.4 CSRF tokens

Every POST form carries a token that must match one stored in the session:

```

def get_csrf_token(request: Request) -> str:
    token = request.session.get(CSRF_SESSION_KEY)
    if not token:
        token = secrets.token_urlsafe(32)
        request.session[CSRF_SESSION_KEY] = token
    return token

def validate_csrf(request: Request, submitted: str | None) -> bool:
    expected = request.session.get(CSRF_SESSION_KEY)
    return bool(expected) and bool(submitted) and secrets.compare_digest(expected, submitted)

```

The token is generated on first use and reused for the session, which is simpler than rotating it per form and enough given the same-site cookie policy.

Handlers that fail validation redirect instead of raising. A stale tab that submits an old token sends the user back to the page rather than to an error.

10.5 Slowing down brute force

The login form is the one endpoint an attacker can hammer. A sliding window per IP address limits it:

```

class LoginRateLimiter:
    def __init__(self, max_attempts: int = 8, window_seconds: int = 900):
        ...
    def _prune(self, ip: str, now: float) -> list[float]:
        recent = [t for t in self._fails.get(ip, []) if now - t < self.window]
        self._fails[ip] = recent
        return recent

```

Eight failures in fifteen minutes and the form answers 429. Only failures are recorded, so a legitimate user is never affected. The state is a dictionary protected by a lock, which is appropriate for a single-process application; a restart clears it, and that is an acceptable trade for having no extra service to run.

Behind a proxy the socket address is the proxy, so the client address is read from `X-Forwarded-For` when present.

10.6 Response headers

One middleware sets the same headers on every response:

```
response.headers["X-Content-Type-Options"] = "nosniff"
response.headers["X-Frame-Options"] = "DENY"
response.headers["Referrer-Policy"] = "no-referrer"
response.headers["Permissions-Policy"] = "geolocation=(), microphone=(), camera=()"
response.headers["Content-Security-Policy"] = (
    "default-src 'self'; "
    "script-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net https://unpkg.com; "
    ...
    "connect-src 'self'; frame-ancestors 'none'; base-uri 'self'")
if settings.cookie_secure:
    response.headers["Strict-Transport-Security"] = "max-age=31536000; includeSubDomains"
```

X-Frame-Options: DENY prevents clickjacking, nosniff stops content type guessing, and the CSP restricts where scripts and styles may come from. The policy still allows 'unsafe-inline' for scripts, because the chart initialisation lives in the templates; that is the weakest line in the policy and the honest place to improve it later.

HSTS is only sent when the deployment is on HTTPS, since sending it from a plain HTTP development server would pin the browser to a scheme the server does not speak.

10.7 Uploads and pasted text

Statement uploads are limited in three ways: an eight megabyte cap, an accepted extension per broker, and a parser that never raises. The size check happens after reading, which is acceptable for a single-user application where the only uploader is the owner.

The pasted-arithmetic parser described in chapter 4 is a security control as much as a convenience: it is the reason a line of text from a phone cannot run code.

10.8 Secrets and the repository

The last control is about the repository rather than the code. .env is ignored by git and only .env.example is committed, with placeholder values. Nothing in the source contains a name, an email, an account or a path belonging to a real person; every one of those comes from the environment.

The API documentation is hidden when IS_PROD=true, which removes a free map of the application from anyone who finds the URL.

The pattern worth copying

Each control here is a few lines. What makes them effective is that they are applied in one place: one middleware for headers, one function for CSRF, one settings object for secrets. Security spread across handlers is security that will be forgotten in the next handler.

Testing

11.1 What is worth testing

The project has around 118 tests, and they are not spread evenly. They concentrate on three kinds of code:

1. **Arithmetic that decides money.** Profit per broker, percentages, the paste parser, aggregation over ranges.
2. **Parsing of external formats.** Broker files that the project does not control and cannot fix.
3. **Contracts that are easy to break by accident.** The health endpoints, the task token, what the backup must never contain.

There are no tests asserting that a template contains a particular heading. Those break on every wording change and catch nothing.

11.2 The database fixture

Most tests run against SQLite in memory, created and destroyed per test:

```
@pytest.fixture
def db():
    engine = create_engine("sqlite:///memory:", connect_args={"check_same_thread": False})
    Base.metadata.create_all(engine)
    s = sessionmaker(bind=engine)()
    try:
        yield s
    finally:
        s.close()
```

Tests that go through TestClient need one extra detail:

```
engine = create_engine("sqlite:///memory:",
                       connect_args={"check_same_thread": False},
                       poolclass=StaticPool)
```

StaticPool forces every connection to be the same one. Without it, the test client runs the request in another thread, gets a fresh connection, and an in-memory database that is empty, because each connection to :memory: is a different database.

A test failing for the right reason

An early version of the security test used the developer's local dev.db by accident. It passed on that machine and failed on a clean clone with no such table: users. The fixture was rewritten to inject an in-memory database through dependency_overrides, and a comment in the test records why.

11.3 Overriding dependencies

FastAPI dependency injection makes it easy to replace the database and the authenticated user:

```
app.dependency_overrides[get_session] = lambda: db
app.dependency_overrides[require_user] = lambda: user
monkeypatch.setattr(inv_router, "validate_csrf", lambda *a, **k: True)
```

The CSRF check is patched because these tests exercise the handler logic, not the CSRF mechanism, which has its own dedicated tests. Being explicit about that split keeps each test focused on one thing.

11.4 No network in the test suite

Three parts of the application talk to the internet: prices, ISIN lookup and Telegram. All three are patched at the boundary.

```
monkeypatch.setattr(prices, "fetch_price", lambda s: Decimal("50000"))
```

```
async def fake_send(chat_id, text, reply_markup=None):
    msgs.append((chat_id, text, reply_markup))
monkeypatch.setattr(bot, "send_message", fake_send)
```

Capturing the outgoing messages in a list turns "the bot replied correctly" into a plain assertion on strings, and the whole suite runs offline in seconds.

11.5 Parser tests without private data

The private version of the project tested the parsers against real statements stored outside the repository, and skipped when they were absent. That is convenient and useless to anyone else: on any other machine the tests silently skip.

The public version builds small synthetic statements inside the test file:

```
KRAKEN_CSV = b"""txid,pair,time,type,ordertype,price,cost,fee,vol
1,BTC/EUR,2026-01-02,buy,market,40000,400.00,0.4,0.01
2,BTC/EUR,2026-02-02,buy,market,45000,450.00,0.4,0.01
3,BTC/EUR,2026-03-02,sell,market,50000,250.00,0.2,0.005
"""

def test_kraken_nets_buys_and_sells():
    r = kraken.parse(KRAKEN_CSV)
    btc = next(p for p in r.positions if p.asset == "BTC/EUR")
    assert btc.invested == Decimal("600.00")      # 400 + 450 - 250
    assert btc.quantity == Decimal("0.01500000")
```

The data is fake, the format is real, and the test runs everywhere. It also documents the format better than a paragraph would.

11.6 The startup test

One test does not check behaviour, it protects a property that is easy to lose:

```

HEAVY = ["matplotlib", "reportlab", "openpyxl", "pandas", "pdfplumber", "yfinance"]

PROBE = f"""
import json, sys
sys.path.insert(0, {str(ROOT)!r})
import app.main
print(json.dumps([m for m in {HEAVY!r} if m in sys.modules]))
"""

def test_boot_does_not_import_heavy_libraries():
    out = subprocess.run([sys.executable, "-c", PROBE], capture_output=True,
                        text=True, cwd=ROOT, check=True)
    loaded = json.loads(out.stdout.strip().splitlines()[-1])
    assert loaded == [], (
        f"Importing app.main loads {loaded}. Move those imports inside the "
        "function that uses them (see app/routers/reports.py).")

```

It runs in a clean subprocess because inside pytest those libraries are already loaded by other tests, so checking `sys.modules` in-process would always fail.

The failure message is as important as the assertion: it names the offending modules and points at the fix. A future contributor adding a top-level `import pandas` gets told exactly what to do instead of wondering why an unrelated test broke.

11.7 Regression tests carry their reason

When a bug is fixed, the test that pins it keeps the explanation:

```

def test_donut_drops_negative_slices():
    """A matplotlib pie raises on slices <= 0; donut has to filter them out."""

```

Six months later, that docstring is the difference between deleting an odd test and understanding why it exists.

11.8 Continuous integration

A GitHub Actions workflow runs the suite on every push and pull request:

```

- uses: actions/setup-python@v5
  with:
    python-version: "3.12"
    cache: pip
- run: pip install -r requirements.txt
- run: python -m pytest -q

```

There is no coverage gate and no linter in the pipeline. The suite either passes or it does not, which for a project of this size is the check that carries the information.

Deployment

12.1 Two paths, both supported

The project supports two deployments, and they do not mix:

- **Self-hosted:** Docker Compose brings up the application and its own Postgres on any machine. Nothing depends on a provider.
- **Managed:** a hosting platform for the application plus a managed Postgres. Free tiers work, with the caveats of chapter 9.

Supporting both is not indecision. The compose file is an escape hatch: if a free tier changes its terms, moving is one command plus restoring the last backup, not a project.

12.2 The image

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8080}"]
```

Three details do useful work. Copying `requirements.txt` before the source means the dependency layer is cached and a code change does not reinstall everything. `PYTHONUNBUFFERED` makes logs appear immediately instead of being buffered until the process ends. And the command reads `$PORT` with a default, because some platforms assign the port at runtime while others expect a fixed one.

12.3 Compose

```
services:
  db:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U finance -d finance"]
      interval: 5s
      retries: 10
  app:
    build: .
    env_file: .env
    environment:
      DATABASE_URL: postgresql+psycopg2://finance:${POSTGRES_PASSWORD:-finance}@db:5432/finance
      USE_SCHEDULER: "true"
    depends_on:
      db:
```

```
condition: service_healthy
```

The healthcheck plus `condition: service_healthy` is the part that matters. Without it, the application starts as soon as the database container exists, which is several seconds before Postgres accepts connections, and the first boot fails on a connection error.

The compose file also overrides `DATABASE_URL` and turns the in-process scheduler on, because this deployment does not sleep and does not need an external cron.

12.4 Managed hosting

`render.yaml` and `fly.toml` are committed as blueprints. Both keep secrets out of the repository: values marked `sync: false` are filled in the provider dashboard.

```
envVars:
- key: IS_PROD
  value: "true"
- key: COOKIE_SECURE
  value: "true"
- key: USE_SCHEDULER
  value: "false"
- key: SECRET_KEY
  sync: false
- key: DATABASE_URL
  sync: false
```

The three non-secret values encode the operating mode of that platform: it serves HTTPS, so the cookie can be secure; it sleeps, so the scheduler is off. The Fly configuration does the opposite, keeping one machine always on and running the migration as a release command.

12.5 Connecting to Postgres

The connection string needs two things that are easy to miss:

```
DATABASE_URL=postgresql+psycopg2://user:PASSWORD@host:5432/dbname?sslmode=require
```

The `+psycopg2` suffix tells SQLAlchemy which driver to use; without it the URL may select a driver that is not installed. `sslmode=require` encrypts the connection, which matters because the database is reached over the public internet.

The engine is configured for a pooled endpoint:

```
def _engine_kwargs() -> dict:
    if settings.is_sqlite:
        return {"connect_args": {"check_same_thread": False}}
    return {"pool_pre_ping": True, "pool_recycle": 1800}
```

`pool_pre_ping` tests a pooled connection before handing it out, and `pool_recycle` discards connections older than thirty minutes. Both exist because a connection pooler, or an idle timeout on the provider side, can close a socket that the application still believes is open. Without these two options the first request after an idle period fails with a confusing error.

12.6 Migrations in production

Migrations do not run automatically on every deploy, except on Fly where a release command does it. Elsewhere the sequence after the first deploy is explicit:

```
alembic upgrade head
python -m scripts.seed
```

Running migrations automatically on every start is convenient and risky: several instances booting at once can run them concurrently, and a failed migration takes the deployment with it. For a single-user application, running them by hand when a release carries one is the cheaper trade.

12.7 The first-boot checklist

1. Generate SECRET_KEY and set it. The application refuses to boot in production without it.
2. Set DATABASE_URL, with the driver prefix and sslmode.
3. Set IS_PROD=true and COOKIE_SECURE=true once HTTPS is live.
4. Run alembic upgrade head and python -m scripts.seed.
5. Set TASKS_TOKEN and register the cron jobs.
6. Point an uptime monitor at /health/db.
7. Configure the Telegram token, chat id, webhook secret and PUBLIC_BASE_URL, then redeploy so the webhook registers.
8. Trigger one backup by hand and check the file arrives.

Step eight is the one people skip. A backup that has never been tested is a belief, not a backup.

Publishing a private project

13.1 The exercise

The application described in the previous chapters runs privately, with real financial data. This repository is a second one, public, built from it.

The goal was not to publish a demo. It was to publish the same application, adapted so that anybody can run their own instance, with nothing personal left anywhere: not in the code, not in the documentation, and not in the commit history.

This chapter describes how that was done, because the process is reusable and most of its traps are not obvious.

13.2 Step one: decide what is personal

Personal data hides in more places than the obvious ones. The audit found five categories.

Where	What was found
Configuration	The owner's full name as the default value of a report setting.
Internal documents	Two reports containing the live application URL, the private repository link and real transaction counts.
Design notes	An internal specification, signed and dated.
Scripts	An importer that read the author's old desktop project from a hardcoded path.
Tests	A test that read the author's real broker statements, and skipped when they were absent.
Parsers	A special case with the ISIN of a product the author owns, plus the names of real people used as text filters.

Table 13.1: What the audit found before publishing.

The first four were removed or replaced. The fifth was rewritten as a generic pattern, which made the parser both shorter and more capable, as chapter 5 described.

The useful question

For each file, ask: *would this still make sense on somebody else's machine?* A path, a name, an ISIN, a live URL and a count of real records all fail that question. It catches more than searching for a name.

13.3 Step two: a fresh history

The public repository was started with `git init`, not by copying the private one.

The reason is that git history is data. A repository that was private for months carries earlier states of every file, including whatever was committed before a name was moved into an environment variable. Removing a secret from the current files does nothing if it is still one git log -p away.

Starting fresh also allowed the history to be rebuilt as a readable narrative: thirteen commits, each one a feature that works, in the order somebody would actually build them.

```
chore: bootstrap FastAPI project skeleton
feat: login with bcrypt, signed session and CSRF
feat: expenses and income with monthly and yearly views
feat: broker statement parsers and P&L in Decimal
feat: bank accounts and net worth snapshots
feat: investment positions, price refresh and ISIN lookup
feat: dashboard with net worth, cashflow and category charts
feat: Excel and PDF reports
feat: Telegram bot to log expenses and pull reports
feat: scheduled tasks, health checks and backups over Telegram
chore: initial Alembic migration
build: Docker image, compose stack and deploy blueprints
docs: README, architecture and deployment guides
```

Building the history that way exposed a dependency problem. The application object in `app/main.py` grows with every feature, so committing the final version early would reference routers that did not exist yet, and committing it late would leave several commits unable to start.

The solution was to reconstruct the intermediate versions: each commit contains the `main.py` that matches the code around it. It costs a few minutes and produces a history where every commit is a working application.

13.4 Step three: the translation

The private application was written in Spanish: comments, docstrings, variable names, database columns, routes and interface text. A public repository in one language and one alphabet is easier for everyone, so everything was translated to English.

The scale is worth knowing before starting: about seven thousand lines of Python and eight hundred of templates.

Before	After
/gastos, /inversiones, /bancos	/expenses, /investments, /banks
importe, categoria, tipo	amount, category, kind
valor_actual, beneficio, saldo	current_value, profit, balance
patrimonio_snapshots	net_worth_snapshots
"gasto", "ingreso" (stored values)	"expense", "income"
/resumen, /ahorro (bot)	/summary, /savings

Table 13.2: Part of the renaming map.

Two lessons came out of it.

Write the map first. Deciding every rename up front, in a table, turns the work into mechanical application. Deciding name by name while editing produces `current_value` in one file and `value_now` in another.

Rename the stored values too, not just the columns. The kind column held the strings *gasto* and *ingreso*. Renaming the column and leaving the values would have produced a database that reads half in each language, and every comparison in the code would have kept a Spanish string literal in it.

13.5 Step four: collapse the migrations

The private repository had three Alembic revisions, describing a history that included a table for a shared property that no longer exists.

A public repository has no deployment to upgrade, so those three were replaced by a single initial revision generated from the translated models. The check that it is correct is two commands:

```
alembic upgrade head      # builds the schema from scratch
alembic check             # no difference between models and database
```

alembic check answering "no new upgrade operations detected" is the proof that the migration and the models agree.

13.6 Step five: rewrite the documentation

The private repository had a stale README describing two features that had been removed months earlier, plus two long internal reports full of live URLs.

None of it was reused. The public repository got a README written against the application as it is today, and two generic guides: an architecture document and a deployment guide. The internal reports were not adapted; they were replaced, because a document written for one person and one server does not become general by deleting the addresses.

13.7 Step six: verify, do not assume

Three checks ran before the repository was considered done.

The whole suite. 118 tests, all passing, after the renaming. A translation this wide is exactly the kind of change that breaks something quietly.

The application, actually running. Not just imports: boot the server, log in, add an expense, see it on the page, download a PDF and a spreadsheet, and check that every page answers 200.

```
curl -s localhost:8011/health          # {"status":"ok",...}
curl -s -o /dev/null -w '%{http_code}' localhost:8011/      # 303 -> /login
# after logging in: /expenses, /investments, /banks, /reports -> 200
# report download                                     -> 200 application/pdf
```

A final search for anything personal. One grep over the whole repository for names, old route words, old domain names and old table names. The only remaining hits should be the ones that are supposed to be there: the repository URL and the copyright line of the licence.

The one that nearly went wrong

The default SSH key on the machine authenticated as a different GitHub account, which had no access to the target one. The first push would have failed with a confusing "repository not found". Checking `ssh -T` and configuring a host alias took two minutes; debugging it

later, with a repository half pushed, would have taken longer.

13.8 A checklist

1. Audit for anything that only makes sense on your machine.
2. Start a new repository; never publish a history that was private.
3. Write the renaming map before renaming anything.
4. Collapse migrations into one initial revision.
5. Rewrite the documentation for a stranger, do not adapt internal notes.
6. Rebuild the history as commits that each work.
7. Run the tests, run the application, then grep for what should not be there.

Lessons

14.1 What the project taught

The chapters above described decisions in place. This one collects the ones that were only obvious afterwards, which are the ones worth passing on.

14.1.1 Correctness in money is a type decision

Choosing Decimal and NUMERIC at the start removed a whole class of bugs for the life of the project. No test had to check that a total was off by a cent, because it cannot be. The equivalent decision made late would have meant touching every model, every parser and every report.

14.1.2 The interface is not the application

Making the Telegram bot call the same services as the web pages meant that adding a command was a few lines and never a second implementation. The rule that route handlers contain no arithmetic is what made that possible, and it is cheap to follow from day one and expensive to retrofit.

14.1.3 Boot time is a feature

Nobody thinks about import time until a platform suspends the process. Then several seconds of loading pandas and matplotlib become the reason a bot looks broken. Moving heavy imports inside functions took an afternoon; a test now keeps them there.

14.1.4 Choose where to fail loudly

The project swallows errors in one place and propagates them in another, and both are deliberate. A price that cannot be fetched leaves the old value alone, because the alternative is a portfolio that stops updating over one delisted ticker. A backup that cannot be delivered returns 500, because a backup that fails quietly is worse than none.

14.1.5 Free tiers are a design constraint

Sleeping applications and pausing databases are not bugs to work around once; they change what the software has to contain. The health endpoints, the split between an in-process scheduler and HTTP task endpoints, and the backup that lives outside the provider all exist because of that constraint.

14.1.6 Error messages should say what to do

The startup test does not say "assertion failed". It names the modules that were loaded and points at the file that shows the correct pattern. Writing that message took an extra minute and will save an hour for whoever hits it.

14.1.7 Small representation changes remove whole bug families

Month ranges kept producing boundary errors until a year and a month became one integer. After that, a range is two comparisons and there are no special cases left to get wrong. When conditions keep multiplying, the representation is usually the problem.

14.2 What would be done differently

- **The content security policy still allows inline scripts**, because chart initialisation lives in the templates. Moving that code to static files would let the policy drop `'unsafe-inline'`.
- **The Trade Republic parser depends on a PDF layout** and will break when the format changes. A statement fixture per layout version would turn that break into a failing test instead of a silent misreading.
- **The login rate limiter lives in memory**, so a restart clears it. Correct for one process, wrong the moment a second one is added.
- **Migrations were reconstructed rather than translated** for the public repository. That was right here, and it would not be an option for a project with real deployments to upgrade.

14.3 Closing

The project began as five desktop scripts that only worked with one computer switched on. It ended as a single application that runs on a small server, answers from a phone, keeps its own history, reports on itself, and survives a free hosting plan without supervision.

The technical work was not the hard part. The hard parts were deciding what not to build, being honest about what fails and how, and making the thing keep working when nobody is watching it. Those are the parts this report tried to make explicit.